

PA1 — 3D Shapes in WebGL

Student ID: 241629

Variant: Pentagonal prism, offset (+0.15, -0.15)

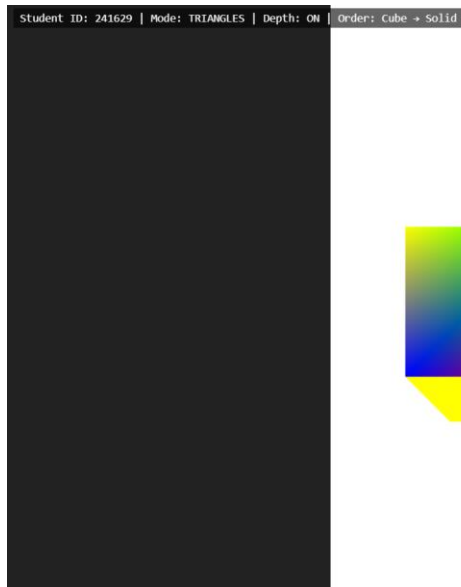
E1 — Depth Testing and Draw Order

Prediction:

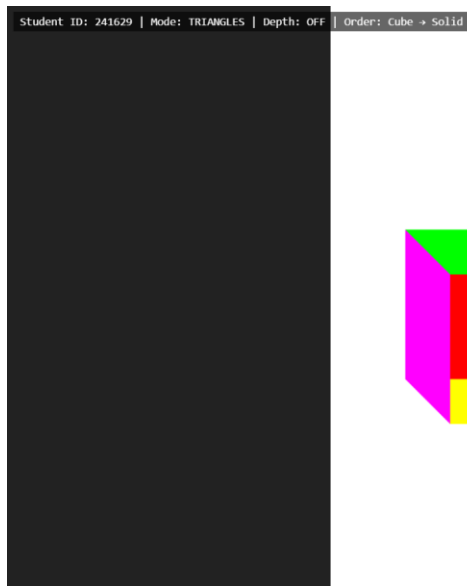
With depth testing ON, the depth buffer should determine which surfaces are visible, so the final image should not depend on the order in which the cube and the prism are drawn. With depth testing OFF, later fragments can overwrite earlier fragments, so the visible faces may depend on the drawing order.

Result:

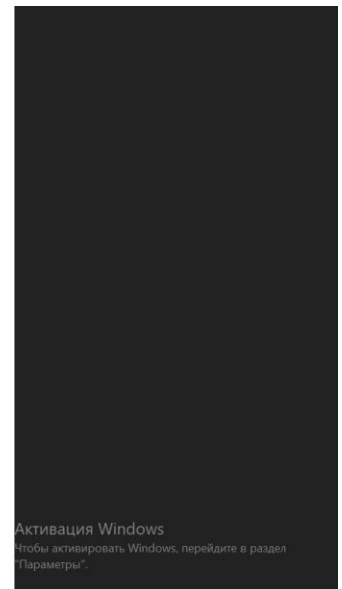
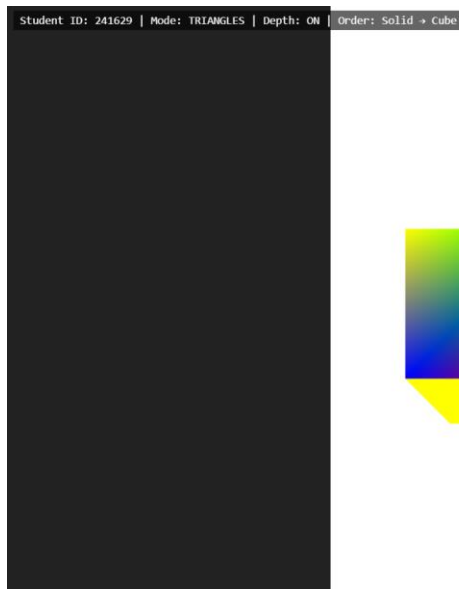
With Depth ON, the cube and pentagonal prism are rendered with hidden surfaces correctly occluded. With Depth OFF, the visible result is affected by the order of the vertices/faces in the position array because later fragments overwrite earlier fragments.



Активация Windows
Чтобы активировать Windows, перейдите в раздел
"Параметры".



Активация Windows
Чтобы активировать Windows, перейдите в раздел
"Параметры".



After pressing S, the draw order changed from Cube → Solid to Solid → Cube, but the final image did not change. This is because the two objects do not overlap, so drawing one before the other does not change the visible pixels.

Overlapping objects

Prediction:

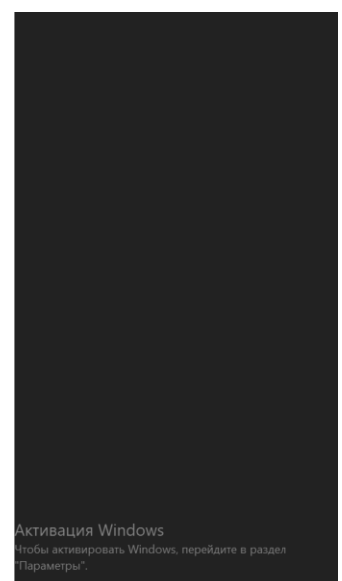
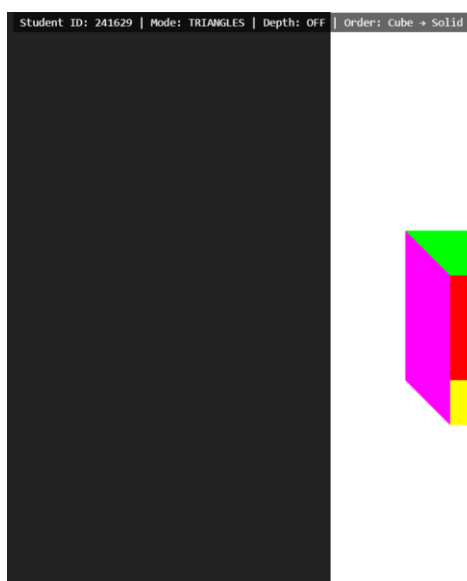
With Depth OFF, changing the draw order should change which object is visible in the overlapping area because later fragments overwrite earlier fragments.

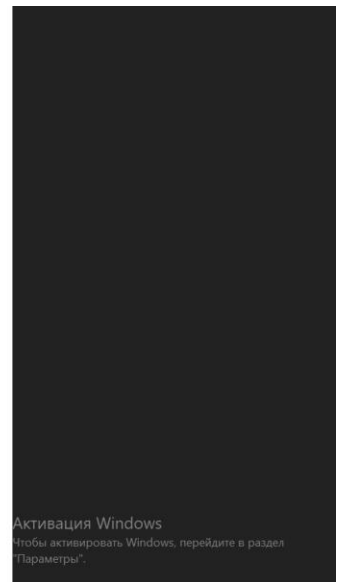
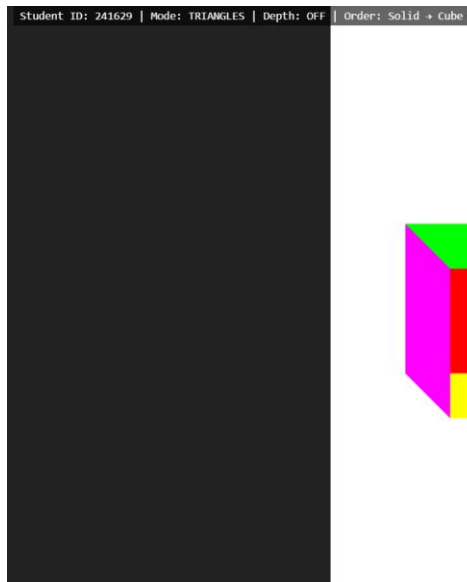
Result:

When the prism was moved to overlap the cube, changing the draw order changed the visible result in the overlapping area.

Explanation:

With depth testing disabled, WebGL does not compare the depth of fragments. The fragments drawn later overwrite the fragments that were drawn earlier. Therefore, changing the order of the two `drawArrays` calls changes which object appears on top in the overlapping region.





E2 — Why negative z is in front

Prediction:

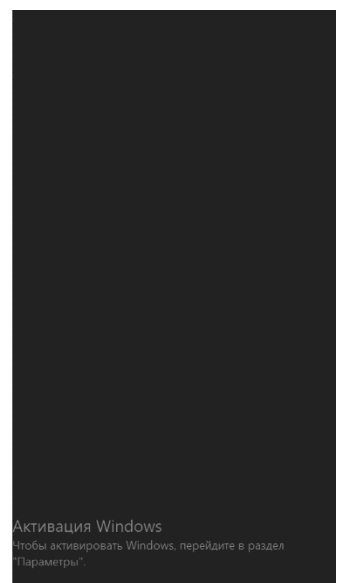
I expected `gl.GREATER` to reject the fragments because the depth buffer is initially cleared to `1.0`, while the generated depth values are normally less than `1.0`.

Result:

After changing the depth function from `gl.LEQUAL` to `gl.GREATER`, both objects disappeared from the scene.

Explanation:

The depth buffer is cleared to `1.0`. With `gl.GREATER`, a new fragment passes only when its depth value is greater than the value already stored in the depth buffer. Since the initial value is `1.0` and the object depth values are smaller, the fragments fail the depth test.



`gl.GREATER + gl.clearDepth(0.0)`

Prediction:

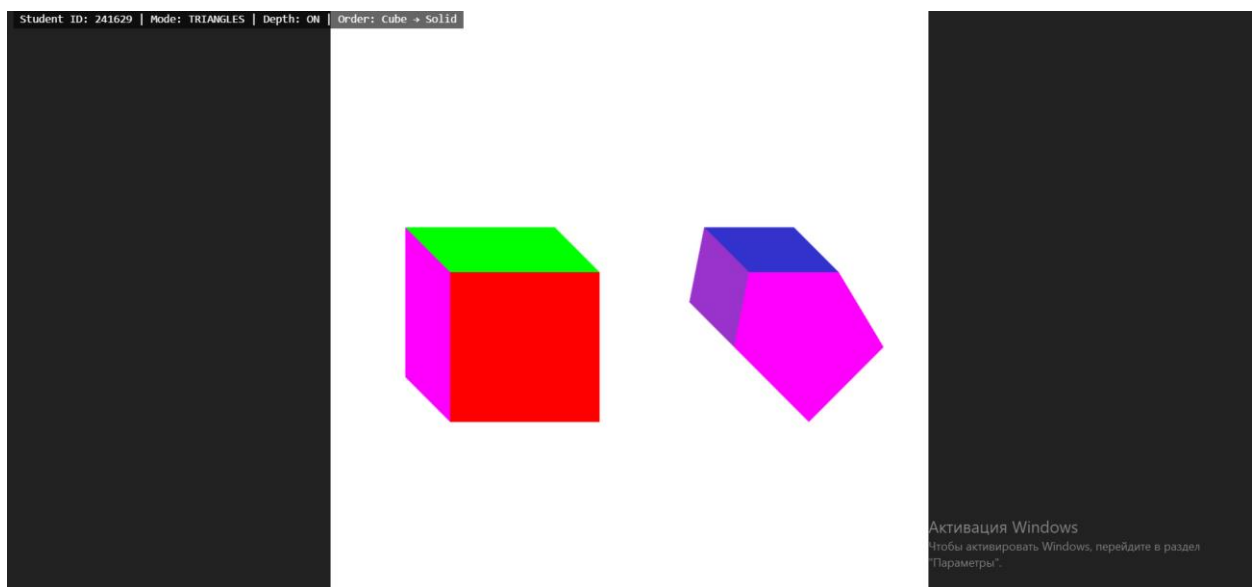
I expected the objects to become visible again because the depth buffer would start at 0.0, so normal depth values greater than 0.0 could pass the GREATER test.

Result:

After adding `gl.clearDepth(0.0)` while keeping `gl.GREATER`, both objects became visible again.

Explanation:

With `gl.GREATER`, a fragment passes when its depth value is greater than the value stored in the depth buffer. Clearing the depth buffer to 0.0 allows the object fragments, whose depth values are greater than 0.0, to pass the test.



E3 — Drawing Modes

Prediction:

I expected the cube to look different in each drawing mode because WebGL interprets the same vertex array differently depending on the selected primitive type.

LINE_LOOP:

`LINE_LOOP` connects each vertex to the next one and also connects the last vertex back to the first vertex. Since the cube is stored as separate triangles, the vertices are not arranged as one continuous outline. Therefore, diagonal lines and an additional closing line appear.

LINES:

`LINES` processes vertices in pairs. The cube contains vertices arranged as independent triangles, so only some of the triangle edges are drawn. The resulting wireframe does not represent all cube edges.

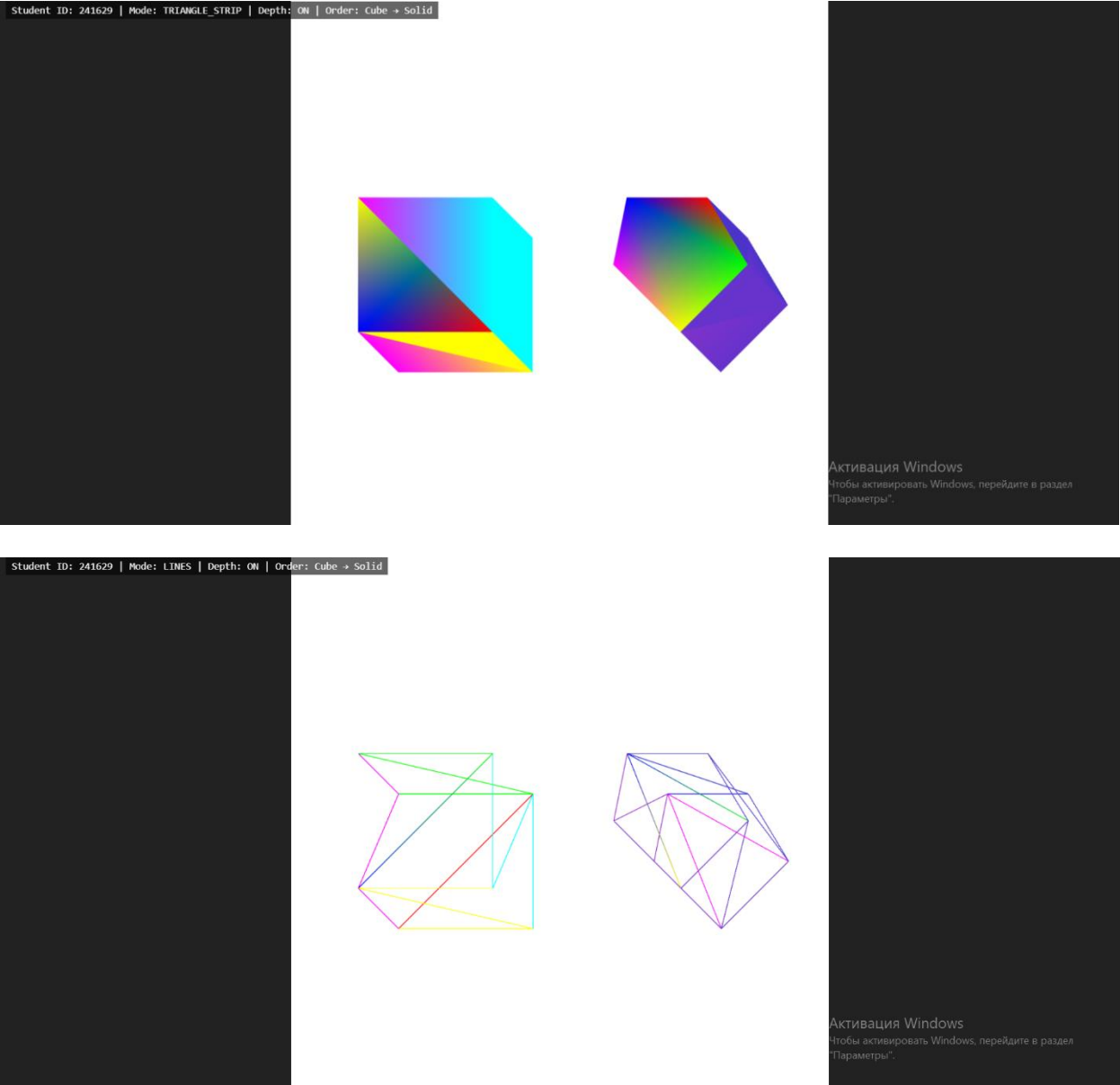
TRIANGLE_STRIP:

`TRIANGLE_STRIP` reuses previous vertices to create consecutive triangles. However, the cube's array was created for independent `TRIANGLES`, where every three vertices form a separate

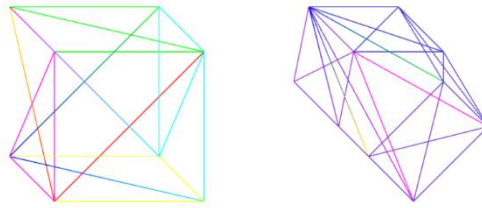
triangle. Therefore, TRIANGLE_STRIP interprets the vertex sequence differently and produces incorrect triangles.

Result:

The screenshots confirm that changing the drawing mode changes how the same vertex data is interpreted and rendered.



Student ID: 241629 | Mode: LINE_LOOP | Depth: ON | Order: Cube → Solid



Активация Windows
Чтобы активировать Windows, перейдите в раздел
"Параметры".

E4 — Missing `bindBuffer`

Prediction:

I expected the geometry to be rendered incorrectly because the position attribute would use the currently bound color buffer instead of the position buffer.

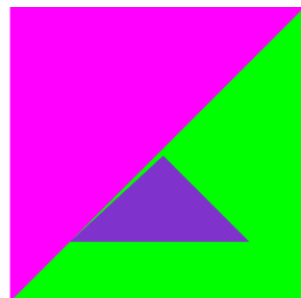
Result:

After removing the `bindBuffer` call before `vertexAttribPointer()` for `aPosition`, the normal cube and prism disappeared and a distorted set of triangles was rendered.

Explanation:

`gl.vertexAttribPointer()` uses the `ARRAY_BUFFER` that is currently bound when the call is made. In this case, the color buffer was still bound, so `aPosition` received its data from the color buffer. The color values were interpreted as vertex positions, producing the distorted geometry. No WebGL error occurred because using a valid buffer with a different attribute layout is still a valid WebGL operation.

Student ID: 241629 | Mode: TRIANGLES | Depth: ON | Order: Cube → Solid



Активация Windows
Чтобы активировать Windows, перейдите в раздел
"Параметры".

E5 — Clipping

Prediction:

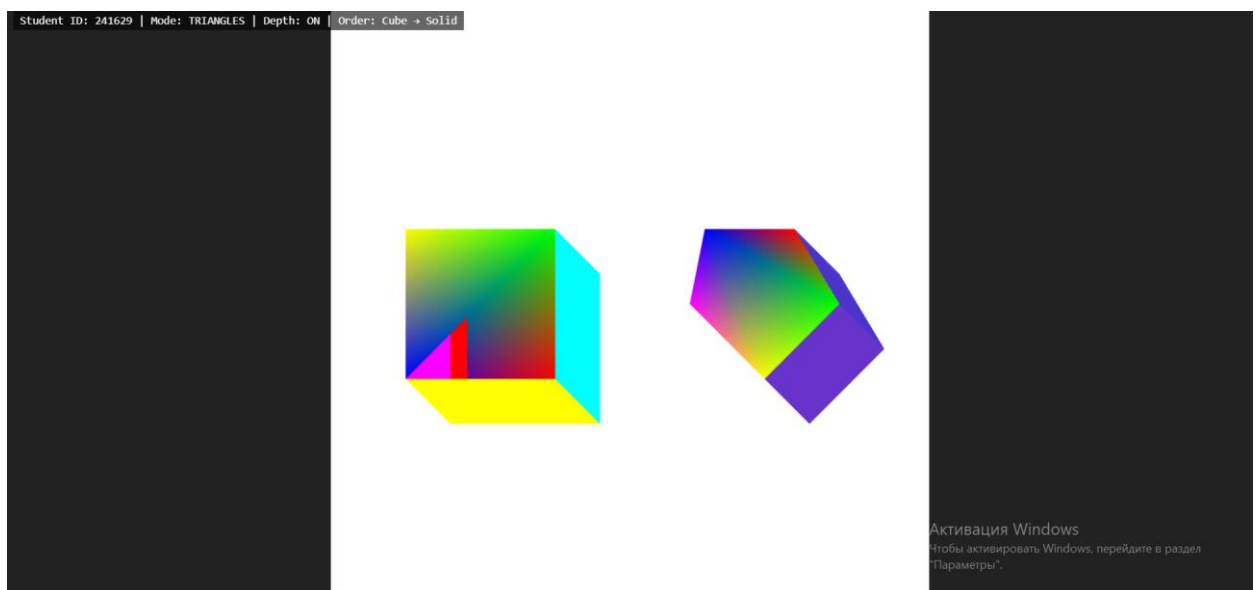
I expected the vertex with $z = 1.2$ to be outside the valid clip-space range and therefore to be clipped.

Result:

After changing one cube vertex from $z = -0.5$ to $z = 1.2$, part of the cube geometry was clipped and the rendered shape changed.

Explanation:

The vertex position is passed directly to `gl_Position`, without any transformation. In clip space, the valid depth range is from -1 to 1 . Since $z = 1.2$ is outside this range, WebGL clips the part of the primitive that lies beyond the clipping boundary.



Prediction:

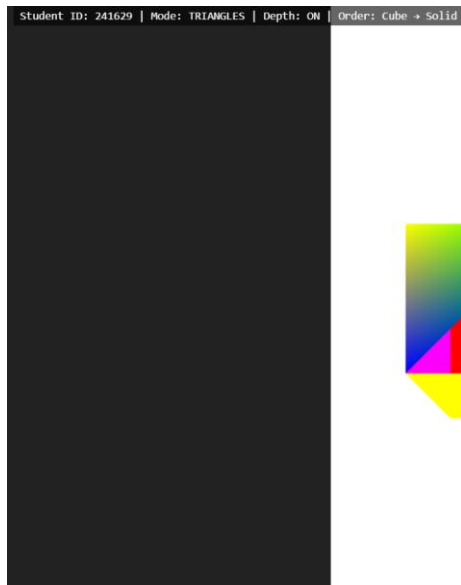
I expected the geometry to be clipped because $x = 1.4$ is outside the valid clip-space range from -1 to 1 .

Result:

After changing the vertex coordinate to $x = 1.4$, the cube geometry changed significantly and the part outside the clip-space boundary was clipped.

Explanation:

Because `gl_Position` receives the vertex coordinates directly, the valid x -coordinate range is -1 to 1 . A vertex with $x = 1.4$ lies outside the right clipping boundary. WebGL therefore clips the primitive against this boundary.



E6 — Buffer Sizes

Result:

The geometry contains 84 vertices in total. The position buffer contains 1008 bytes, and the color buffer contains 1344 bytes. These values were verified using `gl.getBufferParameter(gl.ARRAY_BUFFER, gl.BUFFER_SIZE)`.

Calculation:

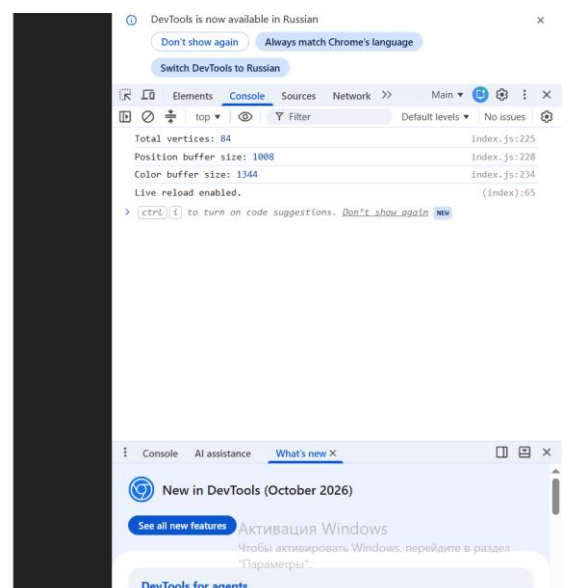
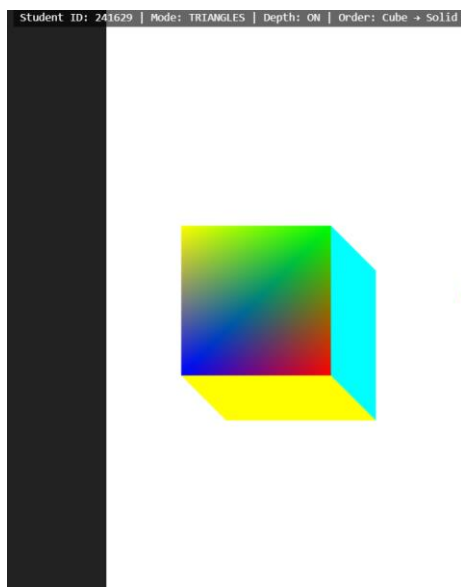
Each vertex has 3 position values stored as 32-bit floats:

$$84 \times 3 \times 4 = 1008 \text{ bytes}$$

Each vertex has 4 color values stored as 32-bit floats:

$$84 \times 4 \times 4 = 1344 \text{ bytes}$$

The calculated values match the values reported by WebGL in the console.



Development Log

During development, I encountered a rendering issue while implementing the WebGL geometry. I checked the vertex data and drawing configuration in the browser and corrected the implementation. The final result was verified in Chrome using the required rendering modes and depth testing. The implementation was committed in b38fbb8.